



Tecnológico de Monterrey

Campus Monterrey

**Evidencia 1 - Actividad Integral de Conceptos Básicos y Algoritmos
Fundamentales (Evidencia Competencia)**

Alejandro Montoya Valdez A01724225

David Cantú Delgado

20 de septiembre de 2026

Importancia de ordenar / buscar en este tipo de logs

Ordenar los registros por fecha y hora nos ayuda a reconstruir cronológicamente los eventos de seguridad. Esto facilita detectar secuencias de ataques, identificar qué ocurrió antes de una falla y consultar periodos específicos sin revisar manualmente todo el archivo.

En el programa, cada fecha se convierte en una clave numérica con el formato AAAAMMDDhhmmss. Una vez ordenados, se localizan los inicios y finales con búsqueda binaria, que cuesta $O(n \log n)$.

Aunque ordenar inicialmente cuesta, es conveniente hacerlo cuando se van a hacer varias consultas, porque evita recorrer los casi 7 mil registros completos en cada búsqueda.

Comparación conceptual

- Swap sort: realiza $O(n^2)$ en todos los casos y puede hacer muchos intercambios. Sencillo, pero no conveniente para archivos grandes. Mejora en datos casi ordenados porque realiza menos intercambios.
- Bubble sort: su peor caso es $O(n^2)$, pero en el mejor de los casos $O(n)$. Funciona mejor con datos casi ordenados.
- Selection sort: siempre busca el elemento menor del resto del arreglo. Su complejidad siempre es $O(n^2)$, sin importar si los datos están casi ordenados.
- Insertion Sort: tiene peor caso $O(n^2)$, pero puede acercarse a $O(n)$ cuando los elementos están casi en su posición correcta. Es una buena opción para conjuntos pequeños o casi ordenados.
- Merge Sort: garantiza $O(n \log n)$. Es estable en esta implementación, porque cuando dos fechas son iguales se toma primero el registro de la mitad izquierda. Su desventaja es el uso de memoria.
- Quick Sort: normalmente $O(n \log n)$, usa poca memoria adicional y puede ser muy rápido. Sin embargo, el programa selecciona el primer elemento como pivote. En datos casi ordenados esto genera particiones desequilibradas y puede acercarlo a $O(n^2)$.
- Shell Sort: Mucho más rápido (en estas pruebas) que los cuadráticos, pero su complejidad depende de la secuencia de saltos y no es estable.

La estabilidad puede importar porque existen timestamps repetidos. Bubble, Insertion y Merge conservan el orden de registros con la misma fecha, Selection, Swap, Quick y shell no lo garantizan.

Interpretación de mediciones

Algoritmo	Log desordenado	Log casi ordenado
Swap Sort	98.2023 ms	10.9925 ms
Bubble Sort	96.4040 ms	1.0954 ms
Selection Sort	56.1863 ms	58.2619 ms
Insertion Sort	85.8264 ms	0.4639 ms
Merge Sort	4.6306 ms	7.8991 ms
Quick Sort	0.9942 ms	7.4060 ms
Shell Sort	4.5311 ms	4.6471 ms

Aquí podemos ver que Merge, Quick y Shell fueron más rápidos con datos desordenados, mientras que insertion y bubble mejoraron mucho con datos casi ordenados. Selection casi no cambio porque siempre tiene $O(n^2)$. Quick sort empeoró con el archivo casi ordenado porque usa el primer elemento como pivote, acercándose al peor caso $O(n^2)$. Los tiempos pueden variar.

Uso de IA

No utilicé mucho IA, mi único prompt fue:

“Me está fallando este quick sort, qué está pasando? No me digas que hacer, solo muéstrame mis errores”.

Lo escribí así para no decirme exactamente cómo arreglarlo, sino que tenía mal, ya que no encontraba el error, así podía pensar yo en cómo arreglarlo.

La IA en este caso no tuvo errores.

Fuera del código, también me ayudó con el README y unos links en [ReportePruebas.md](#), para que ponga links a todas las fotos de pruebas que tenemos en cada caso, tampoco se equivocó.